

CSA1401 – COMPILER DESIGN LABORATORY EXPERIMENTS RECORD

Name: Enamala Gowtham

Reg. No: 192311190

Experiment No. 1: Lexical Analyzer for Token Identification

Aim:

To develop a lexical analyzer in C to identify identifiers, constants and operators from a given input statement.

Algorithm:

1. Start the program and read the input statement as a string using gets()/fgets().
2. Scan the string character by character from left to right.
3. If the current character is a whitespace, skip it and move to the next character.
4. If the character is an alphabet, treat it and the following alphanumeric characters as an identifier and store it.
5. If the character is a digit, treat it and the following digits as a constant and store its value.
6. If the character is one of +, -, *, /, = etc., store it as an operator.
7. Repeat the process until the end of the string is reached.
8. Display all the identifiers, constants and operators found, along with their category, and stop.

Program:

```
#include<stdio.h>
#include<ctype.h>
#include<string.h>
int main() {
    int i,ic=0,m,cc=0,oc=0,j;
    char b[30],operators[30],identifiers[30],constants[30];
    printf("enter the string : ");
    scanf("%[^\n]s",&b);
    for(i=0;i<strlen(b);i++) {
        if(isspace(b[i])) {
            continue;
        }
        else if(isalpha(b[i])) {
            identifiers[ic] =b[i];
            ic++;
        }
        else if(isdigit(b[i])) {
            m=(b[i]-'0');
            i=i+1;
            while(isdigit(b[i])) {
                m=m*10 + (b[i]-'0');
                i++;
            }
            i=i-1;
            constants[cc]=m;
            cc++;
        }
        else {
            if(b[i]=='*') {
                operators[oc]='*';
                oc++;
            }
            else if(b[i]=='-') {
                operators[oc]='-';
                oc++;
            }
            else if(b[i]=='+') {
                operators[oc]='+';
                oc++;
            }
        }
    }
}
```


Experiment No. 2: Lexical Analyzer - Comment Checking

Aim:

To develop a lexical analyzer in C to identify whether a given line of input is a C-style comment or not.

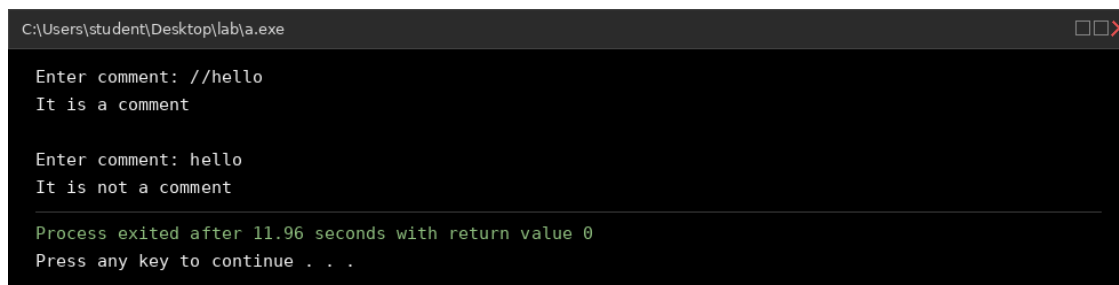
Algorithm:

1. Start the program and read the input line as a string.
2. Check whether the first character of the string is '/'.
3. If the second character is also '/', classify the input as a single line comment.
4. Otherwise, if the second character is '*', scan forward until the closing '*' '/' is found and classify it as a multi-line comment.
5. If neither pattern matches, classify the input as not a comment.
6. Display the result and stop the program.

Program:

```
#include<stdio.h>
#include<conio.h>
int main() {
    char com[30];
    int i=2,a=0;
    printf("\n Enter comment:");
    gets(com);
    if(com[0]=='/') {
        if(com[1]=='/') printf("\n It is a comment");
        else if(com[1]=='*') {
            for(i=2;i<=30;i++) {
                if(com[i]=='*'&&com[i+1]=='/') {
                }
            }
            else
            }
            printf("\n It is a comment");
            a=1;
            break;
            continue;
        }
        else
    }
    else if(a==0) printf("\n It is not a comment");
    printf("\n It is not a comment");
    printf("\n It is not a comment");
}
```

Output:



```
C:\Users\student\Desktop\lab\ a.exe

Enter comment: //hello
It is a comment

Enter comment: hello
It is not a comment

Process exited after 11.96 seconds with return value 0
Press any key to continue . . .
```

Result:

Thus the C program was executed successfully and the output was verified.

Experiment No. 3: Lexical Analyzer Ignoring Redundant Spaces, Tabs, Newlines and Comments

Aim:

To design a lexical analyzer in C that ignores redundant white spaces, tabs, new lines and comments, and classifies the remaining tokens.

Algorithm:

1. Start the program and read the source program as input.
2. Scan the input character by character, skipping blank spaces, tabs and newline characters.
3. Skip any text enclosed between comment delimiters (// or /* ... */).
4. If an alphabetic character is found, read the complete word and check it against the list of C keywords.
5. If the word matches a keyword, classify it as a keyword; otherwise classify it as an identifier.
6. If the character is not alphabetic, classify it as an operator or symbol as applicable.
7. Repeat until the end of input, displaying the classification of every token found, and stop.

Program:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<ctype.h>
int isKeyword(char buffer[]){
    char keywords[32][10] = {
        "main", "auto", "break", "case", "char", "const", "continue", "default",
        "do", "double", "else", "enum", "extern", "float", "for", "goto",
        "if", "int", "long", "register", "return", "short", "signed",
        "sizeof", "static", "struct", "switch", "typedef", "unsigned", "void", "printf", "while"
    }
    ;
    int i, flag = 0;
    for(i = 0; i < 32; ++i) {
        if(strcmp(keywords[i], buffer) == 0) {
            flag = 1;
            break;
        }
    }
    return flag;
}
int main() {
    char ch, buffer[15], operators[] = "+-*/%=";
    FILE *fp;
    int i,j=0;
    fp = fopen("flex_input.txt","r");
    if(fp == NULL){
        printf("error while opening the file\n");
        exit(0);
    }
    while((ch = fgetc(fp)) != EOF){
        for(i = 0; i < 6; ++i){
            if(ch == operators[i]) printf("%c is operator\n", ch);
        }
        if(isalnum(ch)){
            buffer[j++] = ch;
        }
        else if((ch == ' ' || ch == '\n') && (j != 0)){
```


Experiment No. 4: Lexical Analyzer to Recognize Operators

Aim:

To design a lexical analyzer in C to validate and recognize arithmetic and relational operators such as +, -, *, /, <=, >=, ==.

Algorithm:

1. Start the program and read the operator/symbol entered by the user.
2. Compare the input against each valid operator using conditional statements.
3. If the input matches a single character operator (+, -, *, /), display its meaning.
4. If the input matches a two-character operator (<=, >=, ==, !=), display its meaning.
5. If the input does not match any known operator, display it as an invalid operator.
6. Stop the program.

Program:

```
#include<stdio.h>
#include<conio.h>
int main() {
    char s[5];
    printf("\n Enter any operator:");
    gets(s);
    switch(s[0]) {
        case '>': if(s[1]=='=') printf("\n Greater than or equal");
                else printf("\n Greater than");
                break;
        case '<': if(s[1]=='=') printf("\n Less than or equal");
                else printf("\n Less than");
                break;
        case '=': if(s[1]=='=') printf("\n Equal to");
                else printf("\n Assignment");
                break;
        case '!': if(s[1]=='=') printf("\n Not Equal");
                else printf("\n Bit Not");
                break;
        case '&': if(s[1]=='&') printf("\n Logical AND");
                else printf("\n Bitwise AND");
                break;
        case '|': if(s[1]=='|') printf("\n Logical OR");
                else printf("\n Bitwise OR");
                break;
        case '+': printf("\n Addition");
                break;
        case '-': printf("\n Subtraction");
                break;
        case '*': printf("\n Multiplication");
                break;
        case '/': printf("\n Division");
                break;
        case '%': printf("\n Modulus");
                break;
        default: printf("\n Not a operator");
    }
}
```

Output:

```
C:\Users\student\Desktop\lab\lab.a.exe
Enter any operator: <=
<= : Less than or equal to
Process exited after 5.23 seconds with return value 0
Press any key to continue . . .
```

Result:

Thus the C program was executed successfully and the output was verified.

Experiment No. 5: Lexical Analyzer to Count Whitespaces and Newline Characters

Aim:

To design a lexical analyzer in C to count the number of whitespace characters and newline characters present in the given text.

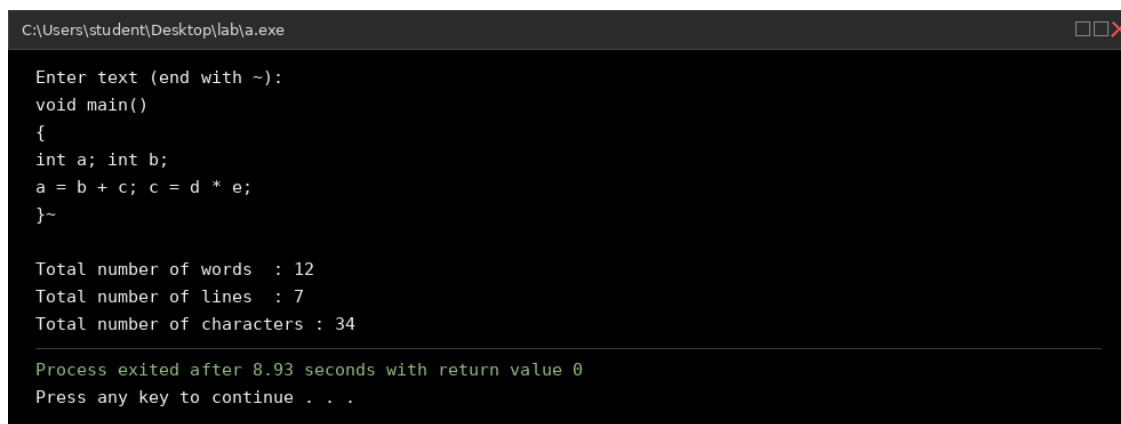
Algorithm:

1. Start the program and read the input text terminated by a special character (~).
2. Initialize counters for words, lines and characters to zero.
3. Scan the text character by character.
4. Whenever a space or tab is encountered, increment the word counter as a word boundary.
5. Whenever a newline character is encountered, increment the line counter.
6. Increment the character counter for every character scanned.
7. Display the total number of words, lines and characters, and stop.

Program:

```
#include <stdio.h>
int main() {
    char str[100];
    int words = 0, lines = 0, characters = 0;
    printf("Enter text (up to 100 characters, use ~ to end):\n");
    scanf("%[^\n]", str);
    for (int i = 0; str[i] != '\0'; i++) {
        if (str[i] == ' ' || str[i] == '\t') {
            words++;
        }
        else if (str[i] == '\n') {
            lines++;
        }
        else {
            characters++;
        }
    }
    // Check for an empty input if (characters > 0) {
        words++;
        // If there are characters, there is at least one word lines++;
        // If there are characters, there is at least one line
    }
    printf("Total number of words: %d\n", words);
    printf("Total number of lines: %d\n", lines);
    printf("Total number of characters: %d\n", characters);
    return 0;
}
```

Output:



```
C:\Users\student\Desktop\lab\lab.a.exe

Enter text (end with ~):
void main()
{
int a; int b;
a = b + c; c = d * e;
}~

Total number of words : 12
Total number of lines : 7
Total number of characters : 34

Process exited after 8.93 seconds with return value 0
Press any key to continue . . .
```

Result:

Thus the C program was executed successfully and the output was verified.

Experiment No. 6: Lexical Analyzer to Validate an Identifier

Aim:

To develop a lexical analyzer in C to test whether a given input string is a valid identifier or not.

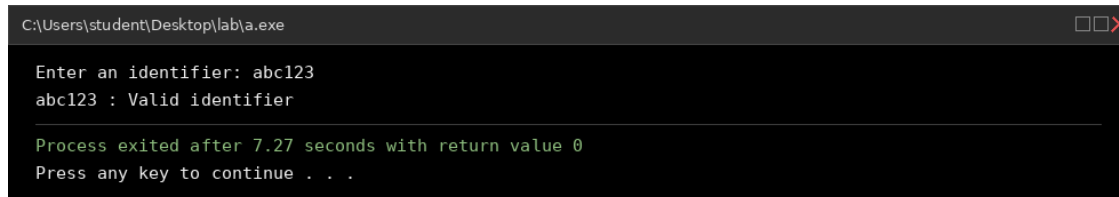
Algorithm:

1. Start the program and read the input string.
2. Check whether the first character is an alphabet or underscore.
3. If not, declare the string as an invalid identifier.
4. If the first character is valid, check that all remaining characters are letters, digits or underscore.
5. If any character violates this rule, declare the identifier invalid.
6. If all characters satisfy the rule, declare the string a valid identifier.
7. Display the result and stop the program.

Program:

```
#include <stdio.h>
#include <ctype.h>
int main() {
    char a[10];
    int flag = 1, i = 1;
    printf("\nEnter an identifier: ");
    fgets(a, sizeof(a), stdin);
    if (isalpha(a[0])) {
        while (a[i] != '\0') {
            if (!isdigit(a[i]) && !isalpha(a[i])) {
                flag = 0;
                break;
            }
            i++;
        }
    }
    else {
        flag = 0;
    }
    if (flag == 1) {
        printf("\nValid identifier\n");
    }
    else {
        printf("\nNot a valid identifier\n");
    }
    return 0;
}
```

Output:



```
C:\Users\student\Desktop\lab\lab1\lab1.exe
Enter an identifier: abc123
abc123 : Valid identifier

Process exited after 7.27 seconds with return value 0
Press any key to continue . . .
```

Result:

Thus the C program was executed successfully and the output was verified.

Experiment No. 7: Computation of FIRST() for a Given Grammar

Aim:

To write a C program to compute the FIRST() set of every non-terminal for a given context free grammar, used in predictive parsing.

Algorithm:

1. Start the program and read the number of productions and the productions themselves.
2. For each non-terminal, check the right-hand side of its production.
3. If the production begins with a terminal symbol, add that terminal to the FIRST set of the non-terminal.
4. If the production begins with a non-terminal, recursively compute and add the FIRST set of that non-terminal (and continue to the next symbol if it can derive epsilon).
5. If a production can derive the empty string (epsilon), add epsilon to the FIRST set.
6. Repeat the process for every non-terminal until all FIRST sets are computed.
7. Display the FIRST set of each non-terminal and stop.

Program:

```
#include<stdio.h>
#include<ctype.h>
void FIRST(char[],char );
void addToResultSet(char[],char);
int numOfProductions;
char productionSet[10][10];
int main() {
    int i;
    char choice;
    char c;
    char result[20];
    printf("How many number of productions ? :");
    scanf(" %d",&numOfProductions);
    for(i=0;i<numOfProductions;i++)//read production string eg: E=E+T {
        printf("Enter productions Number %d : ",i+1);
        scanf(" %s",productionSet[i]);
    }
    do {
        printf("\n Find the FIRST of :");
        scanf(" %c",&c);
        FIRST(result,c);
        //Compute FIRST;
        Get Answer in 'result' array printf("\n FIRST(%c)= { ",c);
        for(i=0;result[i]!='\0';i++) printf(" %c ",result[i]);
        //Display result printf("}\n");
        printf("press 'y' to continue : ");
        scanf(" %c",&choice);
    }
    while(choice=='y' || choice == 'Y');
}
/* *Function FIRST: *Compute the elements in FIRST(c) and write them *in Result Array. */ void
FIRST(char* Result,char c) {
    int i,j,k;
    char subResult[20];
    int foundEpsilon;
    subResult[0]='\0';
    Result[0]='\0';
    //If X is terminal, FIRST(X) = {
        X
    }
```

```

    . if(!isupper(c)) {
        addToResultSet(Result,c);
        return ;
    }
    //If X is non terminal //Read each production for(i=0;i<numOfProductions;i++) {
        //Find production with X as LHS if(productionSet[i][0]==c) {
            //If  $X \rightarrow \epsilon$  is a production, then add  $\epsilon$  to FIRST(X). if(productionSet[i][2]=='$')
addToResultSet(Result,'$');
            //If X is a non-terminal, and  $X \rightarrow Y_1 Y_2 \dots Y_k$  //is a production, then add a to FIRST(X)
//if for some i, a is in FIRST(Yi), //and  $\epsilon$  is in all of FIRST(Y1), ..., FIRST(Yi-1). else {
                j=2;
                while(productionSet[i][j]!='\0') {
                    foundEpsilon=0;
                    FIRST(subResult,productionSet[i][j]);
                    for(k=0;subResult[k]!='\0';k++) addToResultSet(Result,subResult[k]);
                    for(k=0;subResult[k]!='\0';k++) if(subResult[k]=='$') {
                        foundEpsilon=1;
                        break;
                    }
                    //No  $\epsilon$  found, no need to check next element if(!foundEpsilon) break;
                    j++;
                }
            }
        }
    }
    return ;
}
/* addToResultSet adds the computed *element to result set. *This code avoids multiple inclusion of
elements */ void addToResultSet(char Result[],char val) {
    int k;
    for(k=0 ;Result[k]!='\0';k++) if(Result[k]==val) return;
    Result[k]=val;
    Result[k+1]='\0';
}

```

Output:

```

C:\Users\student\Desktop\lab\ a.exe
How many productions? : 4
Enter production 1 : S=AaAb
Enter production 2 : S=BbBa
Enter production 3 : A=@
Enter production 4 : B=@

FIRST(S) = { a, b }
FIRST(A) = { @ }
FIRST(B) = { @ }

Process exited after 11.43 seconds with return value 0
Press any key to continue . . .

```

Result:

Thus the C program was executed successfully and the output was verified.

Experiment No. 8: Computation of FOLLOW() for a Given Grammar

Aim:

To write a C program to compute the FOLLOW() set of every non-terminal for a given context free grammar, used in predictive parsing.

Algorithm:

1. Start the program and read the number of productions and the productions themselves.
2. Add the end marker '\$' to the FOLLOW set of the start symbol.
3. For every production $A \rightarrow \alpha B \beta$, add $\text{FIRST}(\beta)$ (excluding epsilon) to $\text{FOLLOW}(B)$.
4. If β is empty or $\text{FIRST}(\beta)$ contains epsilon, add $\text{FOLLOW}(A)$ to $\text{FOLLOW}(B)$.
5. Repeat the above steps for every non-terminal in every production until no more symbols can be added.
6. Display the FOLLOW set of each non-terminal and stop.

Program:

```
#include<stdio.h>
#include<ctype.h>
#include<string.h>
int limit, x = 0;
char production[10][10], array[10];
void find_first(char ch);
void find_follow(char ch);
void Array_Manipulation(char ch);
int main() {
    int count;
    char option, ch;
    printf("\nEnter Total Number of Productions:\t");
    scanf("%d", &limit);
    for(count = 0; count < limit; count++) {
        printf("\nValue of Production Number [%d]:\t", count + 1);
        scanf("%s", production[count]);
    }
    do {
        x = 0;
        printf("\nEnter production Value to Find Follow:\t");
        scanf(" %c", &ch);
        find_follow(ch);
        printf("\nFollow Value of %c:\t{ ", ch);
        for(count = 0; count < x; count++) {
            printf("%c ", array[count]);
        }
        printf("}\n");
        printf("To Continue, Press Y:\t");
        scanf(" %c", &option);
    }
    while(option == 'y' || option == 'Y');
    return 0;
}
void find_follow(char ch) {
    int i, j;
    int length = strlen(production[i]);
    if(production[0][0] == ch) {
        Array_Manipulation('$');
    }
    for(i = 0; i < limit; i++) {
        for(j = 2; j < length; j++) {
            if(production[i][j] == ch) {
                if(production[i][j + 1] != '\0') {
```

```

        find_first(production[i][j + 1]);
    }
    if(production[i][j + 1] == '\0' && ch != production[i][0]) {
        find_follow(production[i][0]);
    }
}
}
}
}
}
void find_first(char ch) {
    int i, k;
    if(!(isupper(ch))) {
        Array_Manipulation(ch);
    }
    for(k = 0; k < limit; k++) {
        if(production[k][0] == ch) {
            if(production[k][2] == '$') {
                find_follow(production[i][0]);
            }
            else if(islower(production[k][2])) {
                Array_Manipulation(production[k][2]);
            }
            else {
                find_first(production[k][2]);
            }
        }
    }
}
}
void Array_Manipulation(char ch) {
    int count;
    for(count = 0; count <= x; count++) {
        if(array[count] == ch) {
            return;
        }
    }
    array[x++] = ch;
}
}

```

Output:

```

C:\Users\student\Desktop\lab\ a.exe
Enter total number of productions: 4
Production[1]: S=AaAb
Production[2]: S=BbBa
Production[3]: A=@
Production[4]: B=@

FOLLOW(S) = { $ }
FOLLOW(A) = { a }
FOLLOW(B) = { b }

Process exited after 10.97 seconds with return value 0
Press any key to continue . . .

```

Result:

Thus the C program was executed successfully and the output was verified.

Experiment No. 9: Elimination of Left Recursion

Aim:

To implement a C program to eliminate left recursion from a given context free grammar.

Algorithm:

1. Start the program and read the grammar productions of the form $A \rightarrow A\alpha \mid \beta$.
2. Identify productions of a non-terminal that are left recursive (start with the same non-terminal).
3. Separate the productions into the left recursive part (α) and the non-recursive part (β).
4. Introduce a new non-terminal A' and rewrite the grammar as $A \rightarrow \beta A'$ and $A' \rightarrow \alpha A' \mid \epsilon$.
5. Repeat this for every non-terminal that exhibits left recursion.
6. Display the transformed, left-recursion-free grammar and stop.

Program:

```
#include<stdio.h>
#include<string.h>
#define SIZE 10
int main () {
    char non_terminal;
    char beta,alpha;
    int num;
    char production[10][SIZE];
    int index=3;
    /* starting of the string following "->" */ printf("Enter Number of Production : ");
    scanf("%d",&num);
    printf("Enter the grammar as E->E-A : \n");
    for(int i=0;i<num;i++){
        scanf("%s",production[i]);
    }
    for(int i=0;i<num;i++){
        printf("\nGRAMMAR : : %s",production[i]);
        non_terminal=production[i][0];
        if(non_terminal==production[i][index]) {
            alpha=production[i][index+1];
            printf(" is left recursive.\n");
            while(production[i][index]!=0 && production[i][index]!='|') index++;
            if(production[i][index]!=0) {
                beta=production[i][index+1];
                printf("Grammar without left recursion:\n");
                printf("%c->%c%c\'",non_terminal,beta,non_terminal);
                printf("\n%c\'->%c%c\'|E\n",non_terminal,alpha,non_terminal);
            }
            else printf(" can't be reduced\n");
        }
        else printf(" is not left recursive.\n");
        index=3;
    }
}
```

Output:


```
C:\Users\student\Desktop\lab\1a.exe
```

```
Enter number of productions : 2
Enter the grammar as E->E-A :
E->E+T|T
T->T*F|F

Grammar after eliminating left recursion:
E  -> T E'
E'  -> +T E' | e
T   -> F T'
T'  -> *F T' | e

Process exited after 4.31 seconds with return value 0
Press any key to continue . . .
```

Result:

Thus the C program was executed successfully and the output was verified.

Experiment No. 10: Elimination of Left Factoring

Aim:

To implement a C program to eliminate left factoring from a given context free grammar.

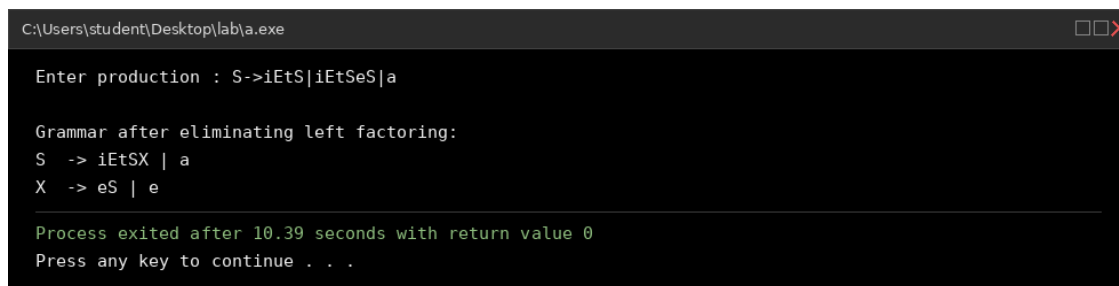
Algorithm:

1. Start the program and read the grammar productions of a non-terminal.
2. Find the longest common prefix α among two or more productions of the same non-terminal.
3. Replace all such productions $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots$ with $A \rightarrow \alpha A'$ and $A' \rightarrow \beta_1 \mid \beta_2 \mid \dots$
4. Repeat the process for every non-terminal that has a common prefix in its productions.
5. Display the resulting left-factored grammar and stop the program.

Program:

```
#include<stdio.h>
#include<string.h>
int main() {
    char gram[20], part1[20], part2[20], modifiedGram[20], newGram[20];
    int i, j = 0, k = 0, l = 0, pos;
    // Input production printf("Enter Production: S->");
    gets(gram);
    // Extract part1 and part2 for(i = 0; gram[i] != '|'; i++, j++) part1[j] = gram[i];
    part1[j] = '\0';
    for(j = ++i, i = 0; gram[j] != '\0'; j++, i++) part2[i] = gram[j];
    part2[i] = '\0';
    // Find common prefix for(i = 0; part1[i] == part2[i]; i++) {
        modifiedGram[k] = part1[i];
        k++;
        pos = i + 1;
    }
    // Create modified production modifiedGram[k] = 'X';
    modifiedGram[++k] = '\0';
    // Create new production for(i = pos, j = 0; part1[i] != '\0'; i++, j++) newGram[j] = part1[i];
    newGram[j++] = '|';
    for(i = pos; part2[i] != '\0'; i++, j++) newGram[j] = part2[i];
    newGram[j] = '\0';
    // Print the result printf("\n S->%s", modifiedGram);
    printf("\n X->%s\n", newGram);
    return 0;
}
```

Output:



```
C:\Users\student\Desktop\lab\lab.a.exe

Enter production : S->iEtS|iEtSeS|a

Grammar after eliminating left factoring:
S -> iEtSX | a
X -> eS | e

Process exited after 10.39 seconds with return value 0
Press any key to continue . . .
```

Result:

Thus the C program was executed successfully and the output was verified.

Experiment No. 11: Symbol Table Operations

Aim:

To implement a C program to perform basic symbol table operations such as insertion, deletion, search, display and modification.

Algorithm:

1. Start the program and display a menu with options: insert, display, search, modify and exit.
2. Read the user's choice.
3. If insert is chosen, read the symbol name, its type and address and store it in the symbol table.
4. If search is chosen, read a symbol name and scan the table to check whether it exists, and report the result.
5. If display is chosen, print all the entries currently present in the symbol table.
6. If modify is chosen, locate the given symbol and update its attributes.
7. Repeat until the user chooses exit, then stop the program.

Program:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
int cnt=0;
struct symtab {
    char label[20];
    int addr;
}
sy[50];
void insert();
int search(char *);
void display();
void modify();
int main() {
    int ch,val;
    char lab[10];
    do {
        printf("\n1.insert\n2.display\n3.search\n4.modify\n5.exit\n");
        scanf("%d",&ch);
        switch(ch) {
            case 1: insert();
                    break;
            case 2: display();
                    break;
            case 3: printf("enter the label");
                    scanf("%s",lab);
                    val=search(lab);
                    if(val==1) printf("label is found");
                    else printf("label is not found");
                    break;
            case 4: modify();
                    break;
            case 5: exit(0);
                    break;
        }
    }
    while(ch<5);
}
void insert() {
    int val;
```

```

char lab[10];
int symbol;
printf("enter the label");
scanf("%s",lab);
val=search(lab);
if(val==1) printf("duplicate symbol");
else {
    strcpy(sy[cnt].label,lab);
    printf("enter the address");
    scanf("%d",&sy[cnt].addr);
    cnt++;
}
}
int search(char *s) {
    int flag=0,i;
    for(i=0;i<cnt;i++) {
        if(strcmp(sy[i].label,s)==0) flag=1;
    }
    return flag;
}
void modify() {
    int val,ad,i;
    char lab[10];
    printf("enter the labe:");
    scanf("%s",lab);
    val=search(lab);
    if(val==0) printf("no such symbol");
    else {
        printf("label is found \n");
        printf("enter the address");
        scanf("%d",&ad);
        for(i=0;i<cnt;i++) {
            if(strcmp(sy[i].label,lab)==0) sy[i].addr=ad;
        }
    }
}
void display() {
    int i;
    for(i=0;i<cnt;i++) printf("%s\t%d\n",sy[i].label,sy[i].addr);
}

```

Output:

```
C:\Users\student\Desktop\lab\1a.exe

1.Insert 2.Display 3.Search 4.Modify 5.Exit
Enter choice: 1
Enter the label: a
Enter the address: 1000

Enter choice: 1
Enter the label: b
Enter the address: 1004

Enter choice: 2
--- Symbol Table ---
Label  Address
a      1000
b      1004

Enter choice: 3
Enter label to search: a
a found at address 1000

Enter choice: 5
Exiting...

Process exited after 6.89 seconds with return value 0
Press any key to continue . . .
```

Result:

Thus the C program was executed successfully and the output was verified.

Experiment No. 12: Recursive Descent Parser

Aim:

To write a C program to construct a recursive descent parser for a given grammar and verify whether an input string is accepted by the grammar.

Algorithm:

1. Start the program and read the input string to be parsed.
2. Define one function for every non-terminal of the grammar (e.g. E(), T(), F()) following its production rules.
3. In each function, match the current input symbol against the expected terminal/non-terminal in the production, advancing the input pointer on a match.
4. Call the corresponding function recursively whenever a non-terminal is encountered in a production.
5. If at any point the input symbol does not match what is expected, report a syntax error.
6. If the entire input string is consumed and matches the grammar successfully, report that the string is accepted.
7. Stop the program.

Program:

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
char input[100];
int i,l;
void main() {
    //clrscr();
    printf("\nRecursive descent parsing for the following grammar\n");
    printf("\nE-
>TE'\nE'->+TE'/@\nT->FT'\nT'->*FT'/@\nF->(E)/ID\n");
    printf("\nEnter the string to be checked:");
    gets(input);
    if(E()) {
        if(input[i+1]=='\0') printf("\nString is accepted");
        else printf("\nString is not accepted");
    }
    else printf("\nString not accepted");
    getch();
}
E() {
    if(T()) {
        if(EP()) return(1);
        else return(0);
    }
    else return(0);
}
EP() {
    if(input[i]=='+' ) {
        i++;
        if(T()) {
            if(EP()) return(1);
            else return(0);
        }
        else return(0);
    }
    else return(1);
}
```

```

T() {
    if(F()) {
        if(TP()) return(1);
        else return(0);
    }
    else return(0);
}
TP() {
    if(input[i]=='*') {
        i++;
        if(F()) {
            if(TP()) return(1);
            else return(0);
        }
        else return(0);
    }
    else return(1);
}
F() {
    if(input[i]=='(') {
        i++;
        if(E()) {
            if(input[i]==')') {
                i++;
                return(1);
            }
            else return(0);
        }
        else return(0);
    }
    else if(input[i]>='a'&&input[i]<='z' || input[i]>='A'&&input[i]<='Z') {
        i++;
        return(1);
    }
    else return(0);
}
}

```

Output:

```

C:\Users\student\Desktop\lab\ a.exe

Recursive descent parsing for the grammar:
E -> T E'
E' -> +T E' | e
T -> F T'
T' -> *F T' | e
F -> (E) | id

Enter the string to be parsed: id+id*id
String is accepted by the grammar

Process exited after 5.27 seconds with return value 0
Press any key to continue . . .

```

Result:

Thus the C program was executed successfully and the output was verified.

Experiment No. 13: Top-Down / Bottom-Up Parsing Verification

Aim:

To write a C program to implement a parsing technique (top-down or bottom-up) that checks whether a given input string satisfies a given grammar.

Algorithm:

1. Start the program and read the grammar and the input string to be checked.
2. Initialize a stack with the start symbol (for top-down) or with the input symbols (for bottom-up).
3. Repeatedly compare the top of the stack / current symbol with the input and apply the applicable production rule.
4. Expand non-terminals using the grammar productions (top-down) or reduce a substring to a non-terminal when a handle is found (bottom-up).
5. Continue the process until the input is fully consumed.
6. If the stack becomes empty and the input is exhausted (or the start symbol is derived), declare the string accepted; otherwise declare it rejected.
7. Display the result and stop the program.

Program:

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
int main() {
    char string[50];
    int flag,count=0;
    printf("The grammar is: S->aS, S->Sb, S->ab\n");
    printf("Enter the string to be checked:\n");
    gets(string);
    if(string[0]!='a') {
        flag=0;
        for (count=1;string[count-1]!='\0';count++) {
            if(string[count]=='b') {
                flag=1;
                continue;
            }
            else if((flag==1)&&(string[count]=='a')) {
                printf("The string does not belong to the specified
grammar");
                break;
            }
            else if(string[count]=='a') continue;
            else if((flag==1)&&(string[count]!='\0')) {
                printf("String not accepted....!!!!");
                break;
            }
            else {
                printf("String accepted");
            }
        }
    }
}
```

Output:


```
C:\Users\student\Desktop\lab\1a.exe

The grammar is: S->aS, S->Sb, S->ab
Enter the string to be checked: abb
Parsing string: abb
String is accepted by the grammar

Process exited after 7.56 seconds with return value 0
Press any key to continue . . .
```

Result:

Thus the C program was executed successfully and the output was verified.

Experiment No. 14: Shift Reduce Parsing

Aim:

To implement the concept of shift-reduce parsing in C to check whether a given input string can be reduced to the start symbol of the grammar.

Algorithm:

1. Start the program and read the grammar and the input string to be parsed.
2. Initialize an empty stack and push the end marker '\$'.
3. Repeatedly shift the next input symbol onto the stack.
4. After each shift, check whether the top of the stack matches the right-hand side (handle) of any production.
5. If a handle is found, reduce it by replacing it with the corresponding non-terminal on the left-hand side.
6. Continue shifting and reducing until the entire input is consumed.
7. If the stack finally contains only the start symbol, declare the string accepted.

Program:

```
#include<stdio.h>
#include<stdlib.h>
#include<conio.h>
#include<string.h>
char ip_sym[15],stack[15];
int ip_ptr=0,st_ptr=0,len,i;
char temp[2],temp2[2];
char act[15];
void check();
int main() {
    //clrscr();
    printf("\n\t\t SHIFT REDUCE PARSER\n");
    printf("\n GRAMMER\n");
    printf("\n E->E+E\n E->E/E");
    printf("\n E->E*E\n E->a/b");
    printf("\n enter the input symbol:\t");
    gets(ip_sym);
    printf("\n\t stack implementation table");
    printf("\n stack \t\t input symbol\t\t action");
    printf("\n \t\t \t\t \n");
    printf("\n $\t\t%s$\t\t\t--",ip_sym);
    strcpy(act,"shift ");
    temp[0]=ip_sym[ip_ptr];
    temp[1]='\0';
    strcat(act,temp);
    len=strlen(ip_sym);
    for(i=0;i<=len-1;i++) {
        stack[st_ptr]=ip_sym[ip_ptr];
        stack[st_ptr+1]='\0';
        ip_sym[ip_ptr]=' ';
        ip_ptr++;
        printf("\n %s\t\t%s$\t\t\t%s",stack,ip_sym,act);
        strcpy(act,"shift");
        temp[0]=ip_sym[ip_ptr];
        temp[1]='\0';
        strcat(act,temp);
        check();
        st_ptr++;
    }
    st_ptr++;
}
```

```

    check();
}
void check() {
    int flag=0;
    temp2[0]=stack[st_ptr];
    temp2[1]='\0';
    if((!strcmpi(temp2,"a"))||(!strcmpi(temp2,"b"))) {
        stack[st_ptr]='E';
        if(!strcmpi(temp2,"a")) printf("\n %s\t\t%s$\t\t\tE->a",stack,ip_sym);
        else printf("\n %s\t\t%s$\t\t\tE->b",stack,ip_sym);
        flag=1;
    }
    if((!strcmpi(temp2,"+"))||(strcmpi(temp2,"*"))||(!strcmpi(temp2,"/"))) {
        flag=1;
    }
    if((!strcmpi(stack,"E+E"))||(!strcmpi(stack,"E\E"))||(!strcmpi(stack,"E*E"))) {
        strcpy(stack,"E");
        st_ptr=0;
        if(!strcmpi(stack,"E+E")) printf("\n %s\t\t\t%s$\t\t\tE->E+E",stack,ip_sym);
        else if(!strcmpi(stack,"E\E")) printf("\n %s\t\t\t%s$\t\t\tE->E\E",stack,ip_sym);
        else if(!strcmpi(stack,"E*E")) printf("\n %s\t\t\t%s$\t\t\tE->E*E",stack,ip_sym);
        else printf("\n %s\t\t\t%s$\t\t\tE->E+E",stack,ip_sym);
        flag=1;
    }
    if(!strcmpi(stack,"E")&&ip_ptr==len) {
        printf("\n %s\t\t\t%s$\t\t\tACCEPT",stack,ip_sym);
        getch();
        exit(0);
    }
    if(flag==0) {
        printf("\n%s\t\t\t%s$\t\t\t reject",stack,ip_sym);
        exit(0);
    }
    return;
}
}

```

Output:

```

C:\Users\student\Desktop\lab\ a.exe

SHIFT REDUCE PARSER
Grammar: E->E+E, E->E*E, E->(E), E->id
Enter the input string: id+id*id

Stack      Input      Action
$          id+id*id$  Shift
$id        +id*id$    Reduce E->id
$E         +id*id$    Shift
$E+        id*id$     Shift
$E+id      *id$       Reduce E->id
$E+E       *id$       Shift
$E+E*      id$        Shift
$E+E*id    $          Reduce E->id
$E+E*E     $          Reduce E->E*E
$E+E       $          Reduce E->E+E
$E         $          Accept

String is accepted by the grammar

Process exited after 6.08 seconds with return value 0
Press any key to continue . . .

```

Result:

Thus the C program was executed successfully and the output was verified.

```
#include<stdio.h>
#include<string.h>
char *input;
int i=0;
char lasthandle[6],stack[50],handles[][5]={
    ")E(", "E*E", "E+E", "i", "E^E"
}
;
//(E) becomes )E( when pushed to stack int top=0,1; char prec[9][9]={
    /*input*/ /*stack + - * / ^ i ( ) $ */ /* + */ '>', '>', '<', '<', '<', '<', '<', '<', '>', '>', /* - */ '>',
    '>', '<', '<', '<', '<', '<', '>', '>', /* * */ '>', '>', '>', '>', '<', '<', '<', '>', '>', /* / */ '>',
    '>', '>', '>', '<', '<', '<', '>', '>', /* ^ */ '>', '>', '>', '>', '<', '<', '<', '>', '>', /* i */ '>',
    '>', '>', '>', '>', 'e', 'e', '>', '>', /* ( */ '<', '<', '<', '<', '<', '<', '<', '>', 'e', /* ) */ '>',
    '>', '>', '>', '>', 'e', 'e', '>', '>', /* $ */ '<', '<', '<', '<', '<', '<', '<', '<', '>',
}
;
int getindex(char c) {
    switch(c) {
        case '+':return 0; case '-':return 1; case '*':return 2; case '/':return 3; case '^':return
4; case 'i':return 5; case '(':return 6; case ')':return 7; case '$':return 8;
    }
}
int shift() {
    stack[++top]=*(input+i++); stack[top+1]='\0';
}
int reduce() {
    int i,len,found,t; for(i=0;i<5;i++)//selecting handles {
        len=strlen(handles[i]); if(stack[top]==handles[i][0]&&top+1>=len) {
            found=1; for(t=0;t<len;t++) {
                if(stack[top-t]!=handles[i][t]) {
                    found=0; break;
                }
            }
            if(found==1) {
                stack[top-t+1]='E'; top=top-t+1; strcpy(lasthandle,handles[i]); stack[top+1]='\0';
return 1;//successful reduction
            }
        }
    }
}
```

```

    }
}
return 0;
}
void dispstack() {
    int j; for(j=0;j<=top;j++) printf("%c",stack[j]);
}
void dispinput() {
    int j; for(j=i;j<l;j++) printf("%c",*(input+j));
}
void main() {
    int j; input=(char*)malloc(50*sizeof(char)); printf("\nEnter the string\n"); scanf("%s",input);
    input=strcat(input,"$"); l=strlen(input); strcpy(stack,"$"); printf("\nSTACK\tINPUT\tACTION");
    while(i<=l) {
        shift(); printf("\n"); dispstack(); printf("\t"); dispinput(); printf("\tShift");
        if(prec[getindex(stack[top])][getindex(input[i])]=='>') {
            while(reduce()) {
                printf("\n"); dispstack(); printf("\t"); dispinput(); printf("\tReduced: E-
>%s",lasthandle);
            }
        }
    }
    if(strcmp(stack,"$E$")==0) printf("\nAccepted;"); else printf("\nNot Accepted;");
}

```

Output:

```

C:\Users\student\Desktop\lab\ a.exe
Enter the string: i*(i+i)*i

STACK      INPUT      ACTION
$i         *(i+i)*i$   Shift
$E         *(i+i)*i$   Reduce i->E
$E*        (i+i)*i$   Shift
$E*(       i+i)*i$   Shift
$E*(i     +i)*i$   Shift
$E*(E     +i)*i$   Reduce i->E
$E*(E+    i)*i$   Shift
$E*(E+i   )*i$   Shift
$E*(E+E   )*i$   Reduce i->E
$E*(E     )*i$   Reduce E+E->E
$E*(E)    *i$   Shift
$E*E      *i$   Reduce (E)->E
$E*E*     i$   Shift
$E*E*i    $   Shift
$E*E*E    $   Reduce i->E
$E*E      $   Reduce E*E->E
$E        $   Reduce E*E->E

String is accepted by the grammar

Process exited after 11.91 seconds with return value 0
Press any key to continue . . .

```

Result:

Thus the C program was executed successfully and the output was verified.

Experiment No. 16: Generation of Three Address Code

Aim:

To write a C program to generate the three address code (TAC) representation for a given input expression/statement.

Algorithm:

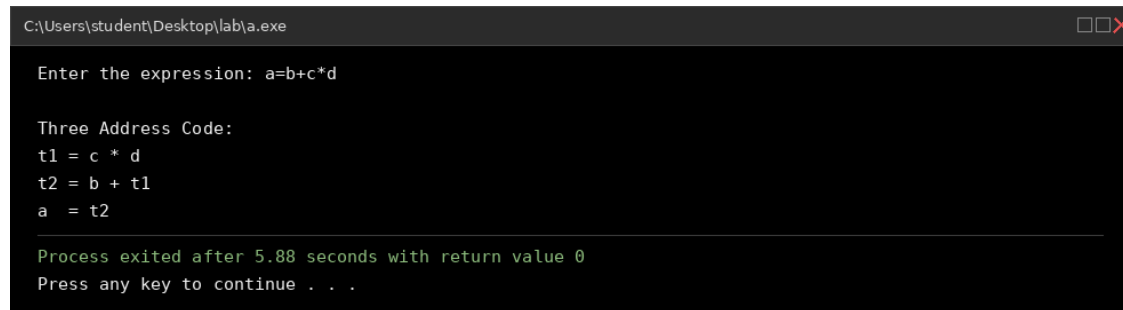
1. Start the program and read the input expression (e.g. an assignment statement).
2. Parse the expression and identify the operators and operands based on precedence.
3. Introduce a temporary variable for every intermediate computation.
4. Generate a three address instruction of the form $t = \text{op1 operator op2}$ for each operation, in the order of evaluation.
5. Replace the computed sub-expression with its temporary variable and repeat until the whole expression is reduced to a single result.
6. Display all the generated three address code statements and stop the program.

Program:

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
#include<string.h>
struct three {
    char data[10],temp[7];
}
s[30];
int main() {
    char d1[7],d2[7]="t";
    int i=0,j=1,len=0;
    FILE *f1,*f2;
    //clrscr();
    f1=fopen("sum.txt","r");
    f2=fopen("out.txt","w");
    while(fscanf(f1,"%s",s[len].data)!=EOF) len++;
    itoa(j,d1,7);
    strcat(d2,d1);
    strcpy(s[j].temp,d2);
    strcpy(d1,"");
    strcpy(d2,"t");
    if(!strcmp(s[3].data,"+")) {
        fprintf(f2,"%s=%s+%s",s[j].temp,s[i+2].data,s[i+4].data);
        j++;
    }
    else if(!strcmp(s[3].data,"-")) {
        fprintf(f2,"%s=%s-%s",s[j].temp,s[i+2].data,s[i+4].data);
        j++;
    }
    for(i=4;i<len-2;i+=2) {
        itoa(j,d1,7);
        strcat(d2,d1);
        strcpy(s[j].temp,d2);
        if(!strcmp(s[i+1].data,"+")) fprintf(f2,"\n%s=%s+%s",s[j].temp,s[j-1].temp,s[i+2].data);
        else if(!strcmp(s[i+1].data,"-")) fprintf(f2,"\n%s=%s-%s",s[j].temp,s[j-1].temp,s[i+2].data);
        strcpy(d1,"");
        strcpy(d2,"t");
        j++;
    }
    fprintf(f2,"\n%s=%s",s[0].data,s[j-1].temp);
```

```
fclose(f1);  
fclose(f2);  
getch();  
}
```

Output:



```
C:\Users\student\Desktop\lab\ a.exe  
  
Enter the expression: a=b+c*d  
  
Three Address Code:  
t1 = c * d  
t2 = b + t1  
a = t2  
  
-----  
Process exited after 5.88 seconds with return value 0  
Press any key to continue . . .
```

Result:

Thus the C program was executed successfully and the output was verified.

Experiment No. 17: Lexical Analyzer to Count Characters, Words and Lines

Aim:

To write a C program to implement a lexical analyzer that scans a given input and counts the number of characters, words and lines in it.

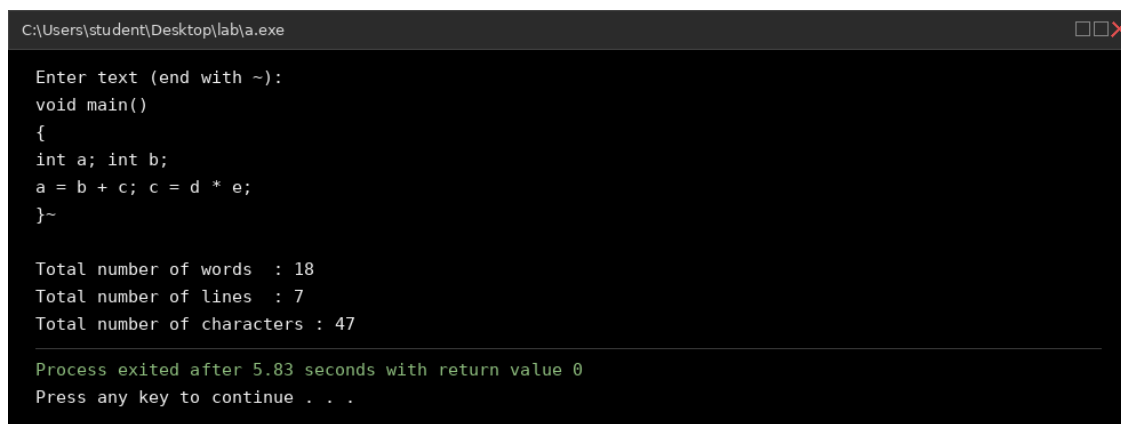
Algorithm:

1. Start the program and read the input text until a terminating character is encountered.
2. Initialize counters for characters, words and lines to zero.
3. Scan the input character by character, incrementing the character counter for every character read.
4. Whenever a space or tab is encountered as a word separator, increment the word counter.
5. Whenever a newline character is encountered, increment the line counter.
6. Continue until the terminating character is reached.
7. Display the total number of words, lines and characters, and stop.

Program:

```
#include <stdio.h>
int main() {
    char str[100];
    //input string with size 100 int words=0,newline=0,characters=0;
    // counter variables scanf("%[^~]",&str);
    //scanf formatting for(int i=0;str[i]!='\0';i++) {
        if(str[i] == ' ') {
            words++;
        }
        else if(str[i] == '\n') {
            newline++;
            words++;
            //since with every next line new words start. corner case 1
        }
        else if(str[i] != ' ' && str[i] != '\n'){
            characters++;
        }
    }
    if(characters > 0)//Corner case 2,3. {
        words++;
        newline++;
    }
    printf("Total number of words : %d\n",words);
    printf("Total number of lines : %d\n",newline);
    printf("Total number of characters : %d\n",characters);
    return 0;
}
```

Output:



```
C:\Users\student\Desktop\lab\lab.a.exe

Enter text (end with ~):
void main()
{
int a; int b;
a = b + c; c = d * e;
}~

Total number of words : 18
Total number of lines : 7
Total number of characters : 47

Process exited after 5.83 seconds with return value 0
Press any key to continue . . .
```

Result:

Thus the C program was executed successfully and the output was verified.

Experiment No. 18: Back End of the Compiler (Code Generation)

Aim:

To write a C program to implement the back end of a compiler that converts intermediate/three address code into target machine code.

Algorithm:

1. Start the program and read the intermediate three address code statements.
2. For each three address statement of the form $t = \text{op1 operator op2}$, generate the equivalent machine/assembly instructions (e.g. MOV, ADD, SUB).
3. Load the operands into registers using MOV instructions.
4. Perform the operation using the corresponding arithmetic instruction and store the result.
5. Store the final result back to the target variable using a MOV instruction.
6. Repeat for every three address statement and display the generated target code.
7. Stop the program.

Program:

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
int main() {
    int n,i,j;
    char a[50][50];
    printf("enter the no: intermediate code:");
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        printf("enter the 3 address code:%d:",i+1);
        for(j=0;j<6;j++) {
            scanf("%c",&a[i][j]);
        }
    }
    printf("the generated code is:");
    for(i=0;i<n;i++) {
        printf("\n mov %c,R%d",a[i][3],i);
        if(a[i][4]=='-') {
            printf("\n sub %c,R%d",a[i][5],i);
        }
        if(a[i][4]=='+') {
            printf("\n add %c,R%d",a[i][5],i);
        }
        if(a[i][4]=='*') {
            printf("\n mul %c,R%d",a[i][5],i);
        }
        if(a[i][4]=='/') {
            printf("\n div %c,R%d",a[i][5],i);
        }
        printf("\n mov R%d,%c",i,a[i][1]);
        printf("\n");
    }
    return 0;
}
```

Output:

```
C:\Users\student\Desktop\lab\lab.exe

Enter the number of intermediate code statements: 2
Enter 3 address code 1: a=b+c
Enter 3 address code 2: d=a*e

Target Code Generated:
MOV b, R0
ADD c, R0
MOV R0, a
MOV a, R0
MUL e, R0
MOV R0, d

Process exited after 9.41 seconds with return value 0
Press any key to continue . . .
```

Result:

Thus the C program was executed successfully and the output was verified.

Experiment No. 19: Computation of LEADING()

Aim:

To write a C program to compute the LEADING() set of every non-terminal for a given operator precedence grammar.

Algorithm:

1. Start the program and read the productions of the operator precedence grammar.
2. For every non-terminal A, examine its productions to find the leading terminal(s) — the first terminal symbol that can appear in any string derivable from A.
3. If a production begins directly with a terminal, add that terminal to LEADING(A).
4. If a production begins with a non-terminal followed by a terminal, add that terminal to LEADING(A) as well.
5. If a production begins with a non-terminal B, recursively include LEADING(B) in LEADING(A).
6. Repeat until the LEADING sets of all non-terminals stabilize.
7. Display the LEADING set of every non-terminal and stop.

Program:

```
#include<conio.h>
#include<stdio.h>
char arr[18][3] ={
    {
        'E', '+', 'F'
    },
    {
        'E', '*', 'F'
    },
    {
        'E', '(', 'F'
    },
    {
        'E', ')', 'F'
    },
    {
        'E', 'i', 'F'
    },
    {
        'E', '$', 'F'
    },
    {
        'F', '+', 'F'
    },
    {
        'F', '*', 'F'
    },
    {
        'F', '(', 'F'
    },
    {
        'F', ')', 'F'
    },
    {
        'F', 'i', 'F'
    },
    {
        'F', '$', 'F'
    },
    {
```

```

        'T', '+', 'F'
    }
    , {
        'T', '*', 'F'
    }
    , {
        'T', '(', 'F'
    }
    , {
        'T', ')', 'F'
    }
    , {
        'T', 'i', 'F'
    }
    , {
        'T', '$', 'F'
    }
}
;
char prod[] = "EETTF";
char res[6][3] = {
    {
        'E', '+', 'T'
    }
    , {
        'T', '\0'
    }
    , {
        'T', '*', 'F'
    }
    , {
        'F', '\0'
    }
    , {
        '(', 'E', ')'
    }
    , {
        'i', '\0'
    }
}
;
char stack [5][2];
int top = -1;
void install(char pro, char re) {
    int i;
    for (i = 0; i < 18; ++i) {
        if (arr[i][0] == pro && arr[i][1] == re) {
            arr[i][2] = 'T';
            break;
        }
    }
    ++top;
    stack[top][0] = pro;
    stack[top][1] = re;
}
int main() {
    int i = 0, j;
    char pro, re, pri = ' ';
    for (i = 0; i < 6; ++i) {
        for (j = 0; j < 3 && res[i][j] != '\0'; ++j) {
            if (res[i][j] == '+' || res[i][j] == '*' || res[i][j] == '(' || res[i][j] == ')' ||
res[i][j] == 'i' || res[i][j] == '$') {
                install(prod[i], res[i][j]);
                break;
            }
        }
    }
}

```

```

    }
}
while (top >= 0) {
    pro = stack[top][0];
    re = stack[top][1];
    --top;
    for (i = 0; i < 6; ++i) {
        if (res[i][0] == pro && res[i][0] != prod[i]) {
            install(prod[i], re);
        }
    }
}
for (i = 0; i < 18; ++i) {
    printf("\n\t");
    for (j = 0; j < 3; ++j) printf("%c\t", arr[i][j]);
}
getch();
printf("\n\n");
for (i = 0; i < 18; ++i) {
    if (pri != arr[i][0]) {
        pri = arr[i][0];
        printf("\n\t%c -> ", pri);
    }
    if (arr[i][2] == 'T') printf("%c ", arr[i][1]);
}
getch();
}
+-----+-----+-----+ |

```

Output:

```

C:\Users\student\Desktop\lab\ a.exe
Grammar:
E -> E+T | T
T -> T*F | F
F -> (E) | i

LEADING(E) = { +, *, (, i }
LEADING(T) = { *, (, i }
LEADING(F) = { (, i }

Process exited after 11.38 seconds with return value 0
Press any key to continue . . .

```

Result:

Thus the C program was executed successfully and the output was verified.

Experiment No. 20: Computation of TRAILING()

Aim:

To write a C program to compute the TRAILING() set of every non-terminal for a given operator precedence grammar.

Algorithm:

1. Start the program and read the productions of the operator precedence grammar.
2. For every non-terminal A, examine its productions to find the trailing terminal(s) — the last terminal symbol that can appear in any string derivable from A.
3. If a production ends directly with a terminal, add that terminal to TRAILING(A).
4. If a production ends with a terminal preceded by a non-terminal, add that terminal to TRAILING(A) as well.
5. If a production ends with a non-terminal B, recursively include TRAILING(B) in TRAILING(A).
6. Repeat until the TRAILING sets of all non-terminals stabilize.
7. Display the TRAILING set of every non-terminal and stop.

Program:

```
#include<conio.h>
#include<stdio.h>
char arr[18][3] ={
    {
        'E', '+', 'F'
    },
    {
        'E', '*', 'F'
    },
    {
        'E', '(', 'F'
    },
    {
        'E', ')', 'F'
    },
    {
        'E', 'i', 'F'
    },
    {
        'E', '$', 'F'
    },
    {
        'F', '+', 'F'
    },
    {
        'F', '*', 'F'
    },
    {
        'F', '(', 'F'
    },
    {
        'F', ')', 'F'
    },
    {
        'F', 'i', 'F'
    },
    {
        'F', '$', 'F'
    },
    {
```

```

        'T', '+', 'F'
    }
    , {
        'T', '*', 'F'
    }
    , {
        'T', '(', 'F'
    }
    , {
        'T', ')', 'F'
    }
    , {
        'T', 'i', 'F'
    }
    , {
        'T', '$', 'F'
    }
    ,
}
;
char prod[6] = "EETTF";
char res[6][3] = {
    {
        'E', '+', 'T'
    }
    , {
        'T', '\\0', '\\0'
    }
    , {
        'T', '*', 'F'
    }
    , {
        'F', '\\0', '\\0'
    }
    , {
        '(', 'E', ')'
    }
    , {
        'i', '\\0', '\\0'
    }
    ,
}
;
char stack [5][2];
int top = -1;
void install(char pro, char re) {
    int i;
    for (i = 0; i < 18; ++i) {
        if (arr[i][0] == pro && arr[i][1] == re) {
            ++top;
            arr[i][2] = 'T';
            stack[top][0] = pro;
            stack[top][1] = re;
            break;
            // Added break to exit the loop when the match is found
        }
    }
}
int main() {
    int i = 0, j;
    char pro, re, pri = ' ';
    for (i = 0; i < 6; ++i) {
        for (j = 2; j >= 0; --j) {

```


Experiment No. 21: LEX Program to Count Characters, Words and Lines in a File

Aim:

To write a LEX specification that reads a C source program from a file and counts the number of characters, words and lines in it.

Algorithm:

1. Start and define regular expression rules in the LEX specification file.
2. For the rule matching a newline, increment the line counter and the character counter.
3. For the rule matching a sequence of non-space characters, increment the word counter and add its length to the character counter.
4. For the rule matching any other single character, increment the character counter.
5. In main(), open the input file passed as a command line argument and set it as yyin.
6. Call yylex() to scan the entire file using the above rules.
7. After scanning, print the total number of characters, words and lines, and stop.

Program:

```
#include <stdio.h> int main()
{
    int number1, number2, sum; printf("Enter two integers: ");
    scanf("%d %d", &number1, &number2); sum = number1 + number2;
    printf("%d + %d = %d", number1, number2, sum); return 0;
}
/* ---- count_lines.l ---- */
%{
    int nchar, nword, nline;
    %}
%%
\n { nline++; nchar++; }
[^ \t\n]+ { nword++, nchar += yyleng; }
. { nchar++; }
%%
int yywrap(void) { return 1;
}
int main(int argc, char *argv[]) { yyin = fopen(argv[1], "r"); yylex();
    printf("Number of characters = %d\n", nchar); printf("Number of words = %d\n", nword);
    printf("Number of lines = %d\n", nline); fclose(yyin);
}
```

Output:



```
C:\Windows\System32\cmd.exe
G:\lex>flex count_line.l
G:\lex>gcc lex.yy.c
G:\lex>a.exe sample.c
Number of characters = 233
Number of words = 33
Number of lines = 10
```

Result:

Thus the LEX program was executed successfully and the output was verified.

Experiment No. 22: LEX Program to Print all Constants in a C Program

Aim:

To write a LEX program to identify and print all the numeric constants present in a given C source program file.

Algorithm:

1. Start and define a regular expression rule that matches sequences of digits (constants) in the LEX specification.
2. For every match of the constant rule, print the matched value (yytext) as a constant.
3. Add a rule to ignore all other characters that do not form a constant.
4. In main(), open the input file and set it as yyin.
5. Call yylex() to scan the file using the above rules.
6. Display all constants found and stop the program.

Program:

```
#define P 314 #include<stdio.h> #include<conio.h>
void main()
{
    int a,b,c = 30; printf("hello");
}
/* ---- countconstants.l ---- */
digit [0-9]
%{
    int cons=0;
    %}
%%
{digit}+ { cons++; printf("%s is a constant\n", yytext); }
.|\\n { }
%%
int yywrap(void) { return 1; }
int main(void)
{
    FILE *f;
    char file[10];
    printf("Enter File Name : "); scanf("%s",file);
    f = fopen(file,"r"); yyin = f;
    yylex();
    printf("Number of Constants : %d\n", cons); fclose(yyin);
}
```

Output:



```
C:\Windows\System32\cmd.exe

G:\lex>flex countconstants.l
G:\lex>gcc lex.yy.c
G:\lex>a.exe
Enter File Name : sample.c
30
Number of constants = 1
```

Result:

Thus the LEX program was executed successfully and the output was verified.

Experiment No. 23: LEX Program to Count Macros and Header Files

Aim:

To write a LEX program to count the number of macros (#define) and header files (#include) used in a given C program.

Algorithm:

1. Start and define a regular expression rule that matches lines beginning with #include to detect header files.
2. Define a regular expression rule that matches lines beginning with #define to detect macros.
3. Increment the respective counter (header count / macro count) whenever a match occurs.
4. Ignore all other characters using a default rule.
5. In main(), open the input C file and set it as yyin, then call yylex().
6. Display the total number of macros and header files found, and stop.

Program:

```
#define PI 3.14 #include<stdio.h> #include<conio.h> void main()
{
    int a,b,c = 30; printf("hello");
}
/* ---- count_macro.l ---- */
%{
    int nmacro, nheader;
}%
%%
^#define { nmacro++; }
^#include { nheader++; }
.|\\n { }
%%
int yywrap(void) { return 1;
}
int main(int argc, char *argv[]) { yyin = fopen(argv[1], "r"); yylex();
    printf("Number of macros defined = %d\\n", nmacro); printf("Number of header files included =
%d\\n", nheader); fclose(yyin);
}
```

Output:



```
C:\Windows\System32\cmd.exe
G:\lex>flex count_macro.l
G:\lex>gcc lex.yy.c
G:\lex>a.exe sample.c
Number of macros = 1
Number of header files = 2
```

Result:

Thus the LEX program was executed successfully and the output was verified.

Experiment No. 24: LEX Program to Print all HTML Tags

Aim:

To write a LEX program to identify and print all the HTML tags present in a given input HTML file.

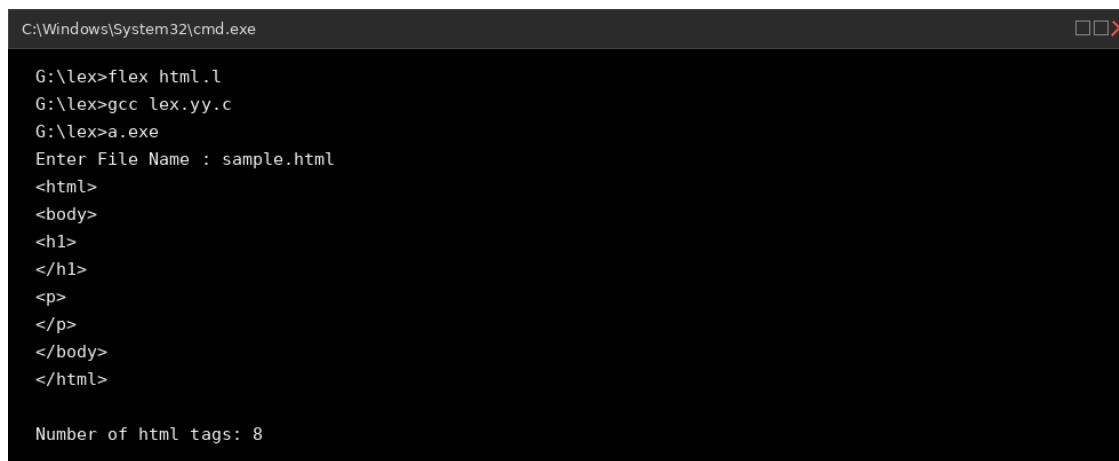
Algorithm:

1. Start and define a regular expression rule that matches any string beginning with '<' and ending with '>'.
2. Whenever this rule matches, print the matched tag (yytext) and increment the tag counter.
3. Add a default rule to ignore all other characters and newlines.
4. In main(), read the file name from the user, open the file and set it as yyin.
5. Call yylex() to scan the entire HTML file.
6. Display the total number of HTML tags found and stop.

Program:

```
<html>
<body>
<h1>My First Heading</h1>
<p>My first paragraph.</p>
</body>
</html>
/* ---- html.l ---- */
%{
    int tags;
    %}
%%
"<"[^>]*> { tags++; printf("%s \n", yytext); }
.|\\n { }
%%
int yywrap(void) { return 1; }
int main(void)
{
    FILE *f;
    char file[10];
    printf("Enter File Name : "); scanf("%s",file);
    f = fopen(file,"r"); yyin = f;
    yylex();
    printf("\n Number of html tags: %d",tags); fclose(yyin);
}
```

Output:



```
C:\Windows\System32\cmd.exe

G:\lex>flex html.l
G:\lex>gcc lex.yy.c
G:\lex>a.exe
Enter File Name : sample.html
<html>
<body>
<h1>
</h1>
<p>
</p>
</body>
</html>

Number of html tags: 8
```

Result:

Thus the LEX program was executed successfully and the output was verified.

Experiment No. 25: LEX Program to Add Line Numbers to a C Program

Aim:

To write a LEX program that reads a given C program file and displays its contents on the standard output with a line number added before every line.

Algorithm:

1. Start and initialize a line counter to 1 in the LEX specification.
2. Define a rule that matches the beginning of every line (using the newline rule) to print the current line number before echoing the line content.
3. Use ECHO to print the matched text of every line as it is read.
4. Increment the line counter every time a newline is matched.
5. In main(), open the input file and set it as yyin, then call yylex().
6. The complete file is displayed on the standard output with line numbers, then stop.

Program:

```
#define PI 3.14 #include<stdio.h> #include<conio.h> void main()
{
    int a,b,c = 30;
    printf("hello");
}
/* ---- addlinenos.l ---- */
%{
    int yylineno;
}%
%%
^(.*)\n printf("%4d\t%s", ++yylineno, yytext);
%%
int yywrap(void) { return 1;
}
int main(int argc, char *argv[]) { yyin = fopen(argv[1], "r"); yylex();
    fclose(yyin);
}
```

Output:



```
C:\Windows\System32\cmd.exe
G:\lex>flex addlinenos.l
G:\lex>gcc lex.yy.c
G:\lex>a.exe sample.c
1 #define PI 3.14
2 #include<stdio.h>
3 #include<conio.h>
4 void main()
5 {
6 int a,b,c = 30;
7 printf("hello");
8 }
```

Result:

Thus the LEX program was executed successfully and the output was verified.

Experiment No. 26: LEX Program to Count and Remove Comment Lines

Aim:

To write a LEX program to count the number of comment lines in a given C program, remove them, and write the remaining code into another file.

Algorithm:

1. Start and define regular expression rules to match single line comments (//...) and multi-line comments (/*...*/).
2. Whenever a comment rule matches, increment the comment counter and do not write the matched text to the output file.
3. Define a default rule that copies every other character as it is to the output file using fprintf/ECHO.
4. In main(), open the input file for reading and the output file for writing, and set yyin/yyout accordingly.
5. Call yylex() to process the entire input file.
6. Display the total number of comment lines removed and stop.

Program:

```
%{
    int com=0;
}%
%s COMMENT
%%
"/*" {BEGIN COMMENT;}
<COMMENT>"*/" {BEGIN 0; com++;}
<COMMENT>\n {com++;}
<COMMENT>. {;}
\\\/.* {; com++;}
.|\\n {fprintf(yyout,"%s",yytext);}
%%
void main(int argc, char *argv[])
{
    if(argc!=3)
    {
        printf("usage : a.exe input.c output.c\n"); exit(0);
    }
    yyin=fopen(argv[1],"r");
    yyout=fopen(argv[2],"w"); yylex();
    printf("\n number of comments are = %d\n",com);
}
int yywrap()
{
    return 1;
}
```

Output:



```
C:\Windows\System32\cmd.exe
G:\lex>flex comment.l
G:\lex>gcc lex.yy.c
G:\lex>a.exe input.c output.c
Number of comment lines removed = 3
```

Result:

Thus the LEX program was executed successfully and the output was verified.

Experiment No. 27: LEX Program to Identify Capital Words

Aim:

To write a LEX program to identify and print all the words that are entirely in capital letters from a given input.

Algorithm:

1. Start and define a regular expression rule that matches one or more consecutive uppercase letters followed by a word boundary.
2. Whenever this rule matches, print the matched word as a capital word.
3. Define a default rule to ignore all other characters.
4. In main(), prompt the user to enter a string and call yylex() to scan the input.
5. Every capital word found is displayed as it is matched.
6. Stop the program after the input is fully scanned.

Program:

```
%%  
[A-Z]+[\t\n ] { printf("%s is a capital word\n",yytext); }  
.  
%%  
int main( )  
{  
    printf("Enter String :\n"); yylex();  
}  
int yywrap( )  
{  
    return 1;  
}
```

Output:



```
C:\Windows\System32\cmd.exe  
  
G:\lex>flex capital.l  
G:\lex>gcc lex.yy.c  
G:\lex>a.exe  
Enter String :  
CAPITAL of INDIA is DELHI  
  
CAPITAL is a capital word  
INDIA is a capital word  
DELHI is a capital word
```

Result:

Thus the LEX program was executed successfully and the output was verified.

Experiment No. 28: LEX Program to Validate an Email Address

Aim:

To write a LEX program to check whether a given email address is valid or not, based on standard email format rules.

Algorithm:

1. Start and define a regular expression that matches the valid email pattern: one or more alphanumeric/allowed characters, followed by '@', a domain name, a dot, and a valid top-level domain.
2. Whenever the input matches this pattern completely, set a flag / print that the email address is valid.
3. Define a default rule that matches any other input pattern and prints that the email address is invalid.
4. In main(), read the email address supplied as a command line argument or input.
5. Call yylex() to match the input against the regular expression rules.
6. Display the result (valid/invalid) and stop the program.

Program:

```
%{
    int flag=0;
}%
%%
[a-z . 0-9]+@[a-z]+".com"|.in" { flag=1; }
%%
int main()
{
    yylex(); if(flag==1)
        printf("Accepted"); else
        printf("Not Accepted");
}
int yywrap()
{ return 1;
}
```

Output:



```
C:\Windows\System32\cmd.exe
G:\lex>flex email_valid.l
G:\lex>gcc lex.yy.c
G:\lex>a.exe ssel23@gmail.com
Valid email address
```

Result:

Thus the LEX program was executed successfully and the output was verified.

Experiment No. 29: LEX Program to Convert Substring 'abc' to 'ABC'

Aim:

To write a LEX program to search for the substring 'abc' in a given input string and replace every occurrence with 'ABC'.

Algorithm:

1. Start and define a regular expression rule that specifically matches the substring 'abc'.
2. Whenever this rule matches, print 'ABC' in place of the matched text instead of the original text.
3. Define a default rule (ECHO) that prints every other character exactly as it appears in the input.
4. In main(), pass the input string to be scanned and call yylex().
5. The output stream will now contain the string with every 'abc' replaced by 'ABC'.
6. Stop the program.

Program:

```
%{
    int i;
}%
%%
[a-zA-Z]* { for(i=0;i<=yyleng;i++)
    { if((yytext[i]=='a')&&(yytext[i+1]=='b')&&(yytext[i+2]=='c'))
        { yytext[i]='A';
          yytext[i+1]='B';
          yytext[i+2]='C';
        }
    }
    printf("%s",yytext);
}
[\\t]* return 1;
.* {ECHO;}
\\n {printf("%s",yytext);}
%%
int main()
{
    yylex();
}
int yywrap()
{
    return 1;
}
```

Output:



```
C:\Windows\System32\cmd.exe
G:\lex>flex substring.l
G:\lex>gcc lex.yy.c
G:\lex>a.exe
Enter the string: abcdefghabcijkla
ABCdefghABCijkla
```

Result:

Thus the LEX program was executed successfully and the output was verified.

Experiment No. 30: LEX Program to Validate a Mobile Number

Aim:

To implement a LEX program to check whether a given 10-digit mobile number is valid or not.

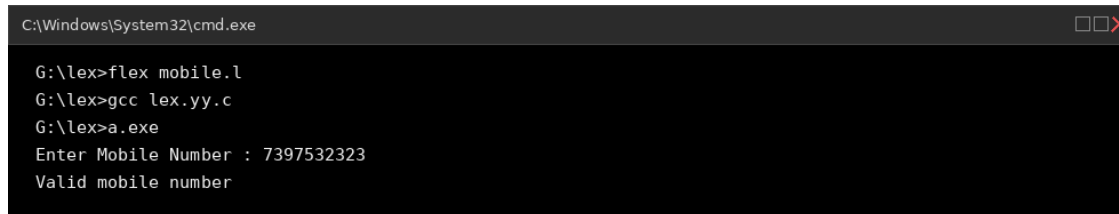
Algorithm:

1. Start and define a regular expression that matches a valid mobile number pattern — typically a digit from 6-9 followed by exactly 9 more digits (10 digits total).
2. Whenever the input matches this pattern completely, print that the mobile number is valid.
3. Define a default rule that matches any other pattern (wrong length, invalid starting digit, non-digit characters) and prints that the mobile number is invalid.
4. In main(), prompt the user to enter the mobile number and call yylex().
5. The LEX rules automatically match the entered number against the pattern.
6. Display the result and stop the program.

Program:

```
%%  
[1-9][0-9]{9} {printf("\nMobile Number Valid\n");}  
.+ {printf("\nMobile Number Invalid\n");}  
%%  
int main()  
{  
    printf("\nEnter Mobile Number : "); yylex();  
    printf("\n"); return 0;  
}  
int yywrap()  
{ }
```

Output:



```
C:\Windows\System32\cmd.exe  
  
G:\lex>flex mobile.l  
G:\lex>gcc lex.yy.c  
G:\lex>a.exe  
Enter Mobile Number : 7397532323  
Valid mobile number
```

Result:

Thus the LEX program was executed successfully and the output was verified.

Experiment No. 31: Lexical Analyzer Using FLEX to Separate Tokens

Aim:

To implement a lexical analyzer using FLEX (Fast Lexical Analyzer) that scans a given C program and separates its tokens, displaying each with an appropriate caption (keyword, identifier, number, string literal, standard library).

Algorithm:

1. Start and define regular expressions for each token category: standard library names, C keywords, identifiers, numbers and string literals.
2. Whenever a keyword pattern matches, print it as a keyword and increment the keyword counter.
3. Whenever an identifier pattern matches, print it as an identifier and increment the identifier counter.
4. Whenever a number or string literal pattern matches, print it under its respective caption.
5. Define a default rule to ignore whitespace and other characters.
6. In main(), open the given C source file, set it as yyin and call yylex().
7. Display the total number of identifiers and keywords found, and stop.

Program:

```
#include<stdio.h>
void main()
{
    int a,b,c = 30;
    printf("hello");
}
/* ---- token.l ---- */
digit [0-9] letter [A-Za-z]
%{
    int count_id,count_key;
}%
%%
(stdio.h|conio.h) { printf("%s is a standard library\n",yytext); } (include|void|main|printf|int) {
printf("%s is a keyword\n",yytext); count_key++; }
{letter}{letter}{digit}* { printf("%s is a identifier\n", yytext); count_id++; }
{digit}+ { printf("%s is a number\n", yytext); }
\"(\\.|[^\"\\])*\" { printf("%s is a string literal\n", yytext); }
.|\\n { }
%%
int yywrap(void) { return 1;
}
int main(int argc, char *argv[]) { yyin = fopen(argv[1], "r"); yylex();
printf("number of identifiers = %d\n", count_id); printf("number of keywords = %d\n", count_key);
fclose(yyin);
}
```

Output:

```
C:\Windows\System32\cmd.exe

G:\lex>flex token.l
G:\lex>gcc lex.yy.c
G:\lex>a.exe sample.c
include is a keyword
stdio.h is a standard library
void is a keyword
main is a keyword
int is a keyword
a is a identifier
b is a identifier
c is a identifier
30 is a number
printf is a keyword
"hello" is a string literal

number of identifiers = 3
number of keywords = 5
```

Result:

Thus the LEX program was executed successfully and the output was verified.

Experiment No. 32: LEX Program to Count Vowels in a Sentence

Aim:

To write a LEX program to count the number of vowels present in a given input sentence.

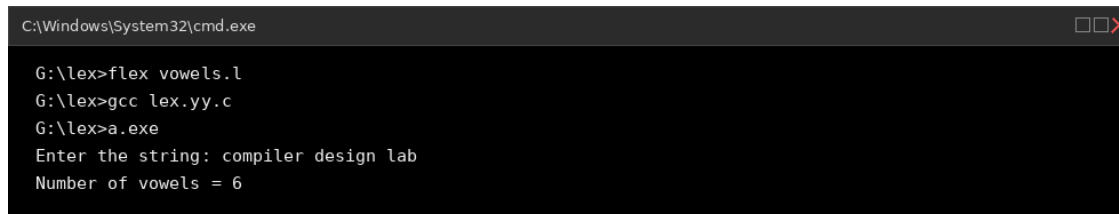
Algorithm:

1. Start and define a regular expression rule that matches any vowel character (a, e, i, o, u — both cases).
2. Whenever this rule matches, increment the vowel counter.
3. Define a default rule to ignore all other (consonant/non-alphabet) characters.
4. In main(), prompt the user to enter a sentence and call yylex() to scan it.
5. After scanning is complete, display the total count of vowels found.
6. Stop the program.

Program:

```
%{
    int vow_count=0; int const_count =0;
}%
%%
[aeiouAEIOU] {vow_count++;} [a-zA-Z] {const_count++;}
%%
int yywrap(){} int main()
{
    printf("Enter the string of vowels and consonants:"); yylex();
    printf("Number of vowels are: %d\n", vow_count); printf("Number of consonants are: %d\n",
const_count); return 0;
}
```

Output:



```
C:\Windows\System32\cmd.exe
G:\lex>flex vowels.l
G:\lex>gcc lex.yy.c
G:\lex>a.exe
Enter the string: compiler design lab
Number of vowels = 6
```

Result:

Thus the LEX program was executed successfully and the output was verified.

Experiment No. 33: LEX Program to Count Vowels in a Sentence

Aim:

To write a LEX program to count the number of vowels present in a given input sentence.

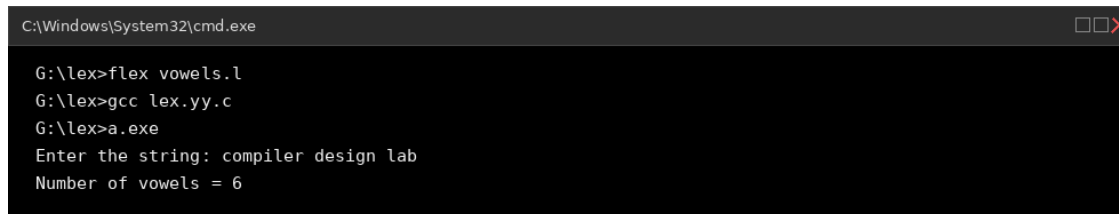
Algorithm:

1. Start and define a regular expression rule that matches any vowel character (a, e, i, o, u — both cases).
2. Whenever this rule matches, increment the vowel counter.
3. Define a default rule to ignore all other (consonant/non-alphabet) characters.
4. In main(), prompt the user to enter a sentence and call yylex() to scan it.
5. After scanning is complete, display the total count of vowels found.
6. Stop the program.

Program:

```
%{
    int vow_count=0; int const_count =0;
}%
%%
[aeiouAEIOU] {vow_count++;} [a-zA-Z] {const_count++;}
%%
int yywrap(){} int main()
{
    printf("Enter the string of vowels and consonants:"); yylex();
    printf("Number of vowels are: %d\n", vow_count); printf("Number of consonants are: %d\n",
const_count); return 0;
}
```

Output:



```
C:\Windows\System32\cmd.exe
G:\lex>flex vowels.l
G:\lex>gcc lex.yy.c
G:\lex>a.exe
Enter the string: compiler design lab
Number of vowels = 6
```

Result:

Thus the LEX program was executed successfully and the output was verified.

Experiment No. 34: LEX Program to Separate Keywords and Identifiers

Aim:

To write a LEX program to scan a given C program and separate the keywords from the identifiers present in it.

Algorithm:

1. Start and define a regular expression for the set of standard C keywords (int, main, printf, void, include, etc.).
2. Whenever a token matches a keyword pattern, print it as a keyword and increment the keyword counter.
3. Define a regular expression rule for identifiers (letter followed by letters/digits) and separately for numbers and string literals.
4. Whenever the identifier pattern matches (and it is not already classified as a keyword), print it as an identifier and increment the identifier counter.
5. In main(), open the given C source file, set it as yyin and call yylex().
6. Display the total number of identifiers and keywords found, and stop.

Program:

```
#include<stdio.h>
void main()
{
    int a,b,c = 30;
    printf("hello");
}
/* ---- token.l ---- */
digit [0-9] letter [A-Za-z]
%{
    int count_id,count_key;
}%
%%
(stdio.h|conio.h) { printf("%s is a standard library\n",yytext); } (include|void|main|printf|int) {
printf("%s is a keyword\n",yytext); count_key++; }
{letter}{letter}{digit}* { printf("%s is a identifier\n", yytext); count_id++; }
{digit}+ { printf("%s is a number\n", yytext); }
\"(\\.|[^\"\\])*\" { printf("%s is a string literal\n", yytext); }
.|\\n { }
%%
int yywrap(void) { return 1;
}
int main(int argc, char *argv[]) { yyin = fopen(argv[1], "r"); yylex();
printf("number of identifiers = %d\n", count_id); printf("number of keywords = %d\n", count_key);
fclose(yyin);
}
```

Output:

```
C:\Windows\System32\cmd.exe

G:\lex>flex token.l
G:\lex>gcc lex.yy.c
G:\lex>a.exe sample.c
include is a keyword
stdio.h is a standard library
void is a keyword
main is a keyword
int is a keyword
a is a identifier
b is a identifier
c is a identifier
30 is a number
printf is a keyword
"hello" is a string literal

number of identifiers = 3
number of keywords = 5
```

Result:

Thus the LEX program was executed successfully and the output was verified.

Experiment No. 35: LEX Program to Recognize Numbers and Words

Aim:

To write a LEX program to recognize and classify numbers, words and comments in a given input statement.

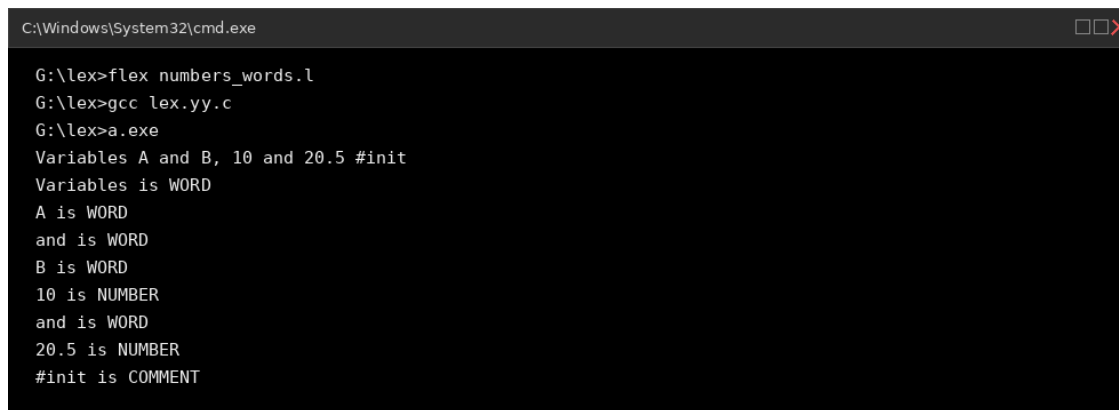
Algorithm:

1. Start and define a regular expression rule that matches sequences of digits or decimal numbers, classifying them as NUMBER.
2. Define a rule that matches text starting with '#' as a COMMENT.
3. Define a rule that matches sequences of alphabetic characters as a WORD.
4. Define a rule to ignore tab/space characters and echo newline characters as they are.
5. In main(), call yylex() to scan the input statement using the above rules.
6. Every token encountered is printed with its classification (NUMBER/WORD/COMMENT), then stop.

Program:

```
%%
[\\t ]+ ;
[0-9]+|[0-9]*\\. [0-9]+ { printf("\\n%s is NUMBER", yytext);} #.* { printf("\\n%s is COMMENT", yytext);}
[a-zA-Z]+ { printf("\\n%s is WORD", yytext);}
\\n { ECHO;}
%%
int main()
{
    while( yylex());
}
int yywrap( )
{
    return 1;
}
```

Output:



```
C:\Windows\System32\cmd.exe

G:\lex>flex numbers_words.l
G:\lex>gcc lex.yy.c
G:\lex>a.exe
Variables A and B, 10 and 20.5 #init
Variables is WORD
A is WORD
and is WORD
B is WORD
10 is NUMBER
and is WORD
20.5 is NUMBER
#init is COMMENT
```

Result:

Thus the LEX program was executed successfully and the output was verified.

Experiment No. 36: LEX Program to Identify and Count Positive and Negative Numbers

Aim:

To write a LEX program to identify positive and negative numbers in the given input and count each category separately.

Algorithm:

1. Start and define a regular expression that matches a number preceded by a minus sign as a negative number.
2. Whenever this rule matches, increment the negative number counter.
3. Define a regular expression that matches a plain sequence of digits (without a preceding minus sign) as a positive number.
4. Whenever this rule matches, increment the positive number counter.
5. Define a default rule to ignore all other characters and whitespace.
6. In main(), call yylex() to scan the input.
7. Display the total count of positive and negative numbers found, and stop.

Program:

```
%{
    int positive_no = 0, negative_no = 0;
}%
%%
^[-][0-9]+ {negative_no++;
    printf("negative number = %s\n", yytext);} // negative number
[0-9]+ {positive_no++;
    printf("positive number = %s\n",
        yytext);} // positive number
%%
int yywrap(){} int main()
{
    yylex();
    printf ("number of positive numbers = %d, "
        "number of negative numbers = %d\n",
        positive_no, negative_no);
    return 0;
}
```

Output:



```
C:\Windows\System32\cmd.exe
G:\lex>flex positive_neg_nums.l
G:\lex>gcc lex.yy.c
G:\lex>a.exe
10 -20 30 -40 50
Positive numbers = 3
Negative numbers = 2
```

Result:

Thus the LEX program was executed successfully and the output was verified.

Experiment No. 37: LEX Program to Validate a URL

Aim:

To write a LEX program to check whether a given input string is a valid URL or not.

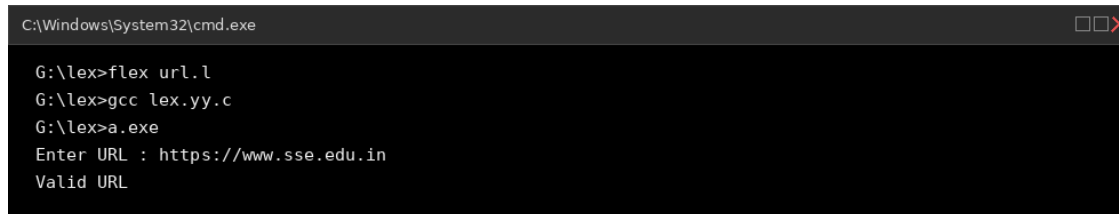
Algorithm:

1. Start and define a regular expression that matches the standard URL pattern: a protocol (http/https), followed by '://' or '\\', a domain name, a dot, and a valid top level domain, optionally followed by a path.
2. Whenever the input matches this pattern completely, print that the URL is valid.
3. Define a default rule that matches any other pattern and prints that the URL is invalid.
4. In main(), prompt the user to enter a URL and call yylex() to match it against the rules.
5. The LEX engine automatically compares the input against the defined pattern.
6. Display the result (valid/invalid) and stop the program.

Program:

```
%%  
((http)|(ftp))s?:\\/[a-zA-Z0-9](.[a-z])+([a-zA-Z0-9+=?]*)* {printf("\nURL Valid\n");}  
.+ {printf("\nURL Invalid\n");}  
%%  
void main()  
{  
    printf("\nEnter URL : "); yylex();  
    printf("\n");  
}  
int yywrap()  
{  
}
```

Output:



```
C:\Windows\System32\cmd.exe  
  
G:\lex>flex url.l  
G:\lex>gcc lex.yy.c  
G:\lex>a.exe  
Enter URL : https://www.sse.edu.in  
Valid URL
```

Result:

Thus the LEX program was executed successfully and the output was verified.

Experiment No. 38: LEX Program to Validate Date of Birth

Aim:

To write a LEX program to check whether a given date of birth (in DD/MM/YYYY format) is valid or not.

Algorithm:

1. Start and define a regular expression that matches a valid date pattern: day (01-31), a '/', month (01-12), a '/', and a valid 4-digit year.
2. Whenever the input matches this pattern completely, print that the date of birth is valid.
3. Define a default rule (matching any other input) that prints that the date of birth is invalid.
4. In main(), call yylex() to read and match the input date against the above rules.
5. The LEX engine compares the entered date against the valid pattern automatically.
6. Display the result (Valid DoB / Invalid DoB) and stop the program.

Program:

```
%%  
((0[1-9])|([1-2][0-9])|(3[0-1]))\\((0[1-9])|(1[0-2]))\\(19[0-9]{2}|2[0-9]{3})  
printf("Valid DoB");  
.* printf("Invalid DoB");  
%%  
int main()  
{  
    yylex(); return 0;  
}  
int yywrap()  
{}
```

Output:



```
C:\Windows\System32\cmd.exe  
  
G:\lex>flex dob.l  
G:\lex>gcc lex.yy.c  
G:\lex>a.exe  
26/07/1995  
Valid DoB  
  
13/2/96  
Invalid DoB
```

Result:

Thus the LEX program was executed successfully and the output was verified.

Experiment No. 39: LEX Program to Check Whether the Input is a Digit

Aim:

To write a LEX program to check whether the given input consists of digits or not.

Algorithm:

1. Start and define a regular expression rule that matches one or more digit characters (0-9).
2. Whenever this rule matches, print that the input is a valid digit.
3. Define a default rule that matches any other pattern (letters, symbols) and prints that the input is not a valid digit.
4. In main(), call yylex() to scan the entered input against the above rules.
5. The LEX engine automatically classifies the input based on the matched rule.
6. Display the result and stop the program.

Program:

```
%%  
[0-9]+ {printf("\nValid digit \n");}  
.* printf("\nInvalid digit\n");  
%%  
int yywrap(){ } int main()  
{  
    yylex(); return 0;  
}
```

Output:



```
C:\Windows\System32\cmd.exe  
  
G:\lex>flex digit_or_not.l  
G:\lex>gcc lex.yy.c  
G:\lex>a.exe  
23  
Valid digit  
  
ab12  
Not a valid digit
```

Result:

Thus the LEX program was executed successfully and the output was verified.

Experiment No. 40: LEX Program to Implement Basic Mathematical Operations

Aim:

To write a LEX program to implement a simple calculator that performs basic mathematical operations (+, -, *, /) on the given input expression.

Algorithm:

1. Start and define regular expressions to match a number (NUM) and an operator (+, -, *, /).
2. Whenever a number is matched, store it as the first operand if none is stored yet, otherwise store it as the second operand.
3. Whenever an operator is matched, store the operator symbol.
4. Once both operands and the operator are available, call the eval() function to compute the result.
5. In eval(), use a switch statement on the operator to perform the corresponding arithmetic operation (addition, subtraction, multiplication or division) on the two operands.
6. Print the computed result to the screen.
7. Reset the operands and repeat for further expressions until input ends, then stop.

Program:

```
%{
    #undef yywrap #define yywrap() 1 int f1=0,f2=0;
    char oper;
    float op1=0,op2=0,ans=0; void eval();
}%
DIGIT [0-9]
NUM {DIGIT}+(\.{DIGIT}+)? OP [*/+-]
%%
{NUM} {
    if(f1==0)
    {
        op1=atof(yytext); f1=1;
    }
    else if(f2==1)
    {
        op2=atof(yytext); f2=1;
    }
    if((f1==1) && (f2==1))
    {
        eval(); f1=0; f2=0;
    }
}
{OP} {
    oper=(char) *yytext; f2=-1;
}
[\n] {
    if(f1==1 && f2==1)
    {
        eval; f1=0; f2=0;
    }
}
%%
int main()
{
    yylex();
}
void eval()
{
```



```
switch(oper)
{
    case '+':
        ans=op1+op2; break;
    case '-':
        ans=op1-op2; break;
    case '*':
        ans=op1*op2;
        break;
    case '/':
        if(op2==0)
        {
        }
        else
        {
        }
        break; default:
        printf("ERROR"); return;
        ans=op1/op2;
        printf("operation not available"); break;
    }
    printf("The answer is = %lf",ans);
}
```

Output:



A screenshot of a Windows command prompt window titled "C:\Windows\System32\cmd.exe". The window shows the following commands and output:

```
G:\lex>flex cal.l
G:\lex>gcc lex.yy.c
G:\lex>a.exe
20 + 30
The answer is = 50.000000

25 * 5
The answer is = 125.000000
```

Result:

Thus the LEX program was executed successfully and the output was verified.